

# MuJoCo 与机器人仿真基础

本文面向四足组后续机器人开发所需的 MuJoCo 基础使用，主要介绍 MuJoCo 的基本作用、Python 接口、MJCF 模型结构、URDF 与 MJCF 的关系、场景与机器人模型的组织方式、关节与执行器、`qpos` / `qvel` / `ctrl`、仿真循环以及常见调试方法。本文不要求读者提前学习机器人动力学，也不会展开接触求解器、雅可比矩阵和强化学习算法等内容。

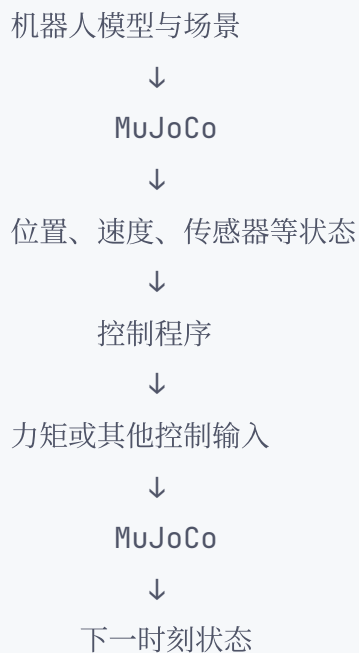
本阶段的目标是先建立一个清晰的机器人仿真程序框架：能够看懂一个基础 MJCF 模型，知道机器人状态和控制量存放在哪里，能够使用 Python 加载模型并推进仿真，并理解后续四足仿真代码中的主要组成部分。完成这些内容后，再进入具体机器人的模型整理和控制程序会顺畅很多。

本文以 MuJoCo 3.x 的官方 Python 接口为主。MuJoCo 仍在持续更新，如果个别接口在未来版本中发生变化，应以官方文档为准。

## 1. MuJoCo 与机器人仿真

MuJoCo (Multi-Joint dynamics with Contact) 是一套面向刚体动力学和接触仿真的物理引擎，在机器人控制、强化学习和运动规划中使用较多。对于四足机器人，MuJoCo 可以根据机器人的质量、惯量、关节结构、碰撞几何和执行器等信息计算机器人在重力、接触和控制输入作用下的运动状态。

一个最基础的机器人仿真过程可以整理为：



控制程序通常会不断读取当前状态，根据控制算法计算新的控制量，再让 MuJoCo 向前推进一个或多个仿真步。后续使用强化学习策略时，神经网络策略也只是控制程序的一部分：它读取观测量并产生动作，仿真器负责根据这些动作计算机器人下一时刻的运动状态。

因此，在目前的培训阶段，需要先把 MuJoCo 当作一个可以被程序调用的物理仿真环境来理解。当前最重要的问题包括模型从哪里加载、状态放在哪里、控制输入写在哪里，以及一次仿真更新是如何发生的。

## 2. 安装与运行环境

MuJoCo 官方提供 Python 包，可以直接通过 `pip` 安装。Ubuntu 中首先确认 Python 可以正常使用：

```
python3 --version
python3 -m pip --version
```

如果系统中没有 `pip`，可以安装：

```
sudo apt update
sudo apt install python3-pip
```

随后安装 MuJoCo:

```
python3 -m pip install mujoco
```

安装完成后可以进入 Python 测试:

```
python3
```

然后输入:

```
import mujoco
print(mujoco.__version__)
```

能够正常输出版本号, 说明 Python 已经能够找到 MuJoCo。官方 Python 包已经包含运行所需的 MuJoCo 库, 对于本阶段的 Python 使用, 不需要再单独安装一套 MuJoCo 动态库。

如果后续同时维护多个 Python 项目, 建议逐步学习虚拟环境, 例如 [venv](#) 或 Conda, 用来隔离不同项目的依赖版本。本阶段如果暂时没有使用虚拟环境, 也应至少记住下面的命令:

```
which python3
python3 -m pip show mujoco
```

它们可以帮助确认当前实际使用的是哪个 Python, 以及 MuJoCo 被安装到了哪里。以后遇到“已经安装但 import 失败”的问题时, 首先应检查 Python 和 pip 是否属于同一个环境。

---

### 3. 第一个 MuJoCo 模型

在直接加载四足机器人之前，先使用一个很小的模型理解 MuJoCo 的基本运行过程。建立目录：

```
mkdir -p ~/mujoco_training/01_falling_box
cd ~/mujoco_training/01_falling_box
```

创建 `scene.xml`：

```
<mujoco model="falling_box">
  <option timestep="0.002" gravity="0 0 -9.81"/>

  <worldbody>
    <light pos="0 0 3"/>

    <geom
      name="floor"
      type="plane"
      size="5 5 0.1"
      rgba="0.8 0.8 0.8 1"
    />

    <body name="box" pos="0 0 1">
      <freejoint/>
      <geom
        type="box"
        size="0.1 0.1 0.1"
        mass="1"
        rgba="0.2 0.5 0.8 1"
      />
    </body>
  </worldbody>
</mujoco>
```

这个模型包含一个地面和一个位于地面上方的方块。 `gravity="0 0 -9.81"` 设置重力， `<freejoint/>` 允许方块在三维空间中自由平移和转动，因此仿真开始后方块会在重力作用下落到地面。

创建 `simulate.py`：

```
import time

import mujoco
import mujoco.viewer

model = mujoco.MjModel.from_xml_path("scene.xml")
data = mujoco.MjData(model)

with mujoco.viewer.launch_passive(model, data) as viewer:
    while viewer.is_running():
        step_start = time.time()

        mujoco.mj_step(model, data)
        viewer.sync()

        time_left = model.opt.timestep - (time.time() - step_start)
        if time_left > 0:
            time.sleep(time_left)
```

运行：

```
python3 simulate.py
```

如果环境正常，会出现 MuJoCo Viewer，并看到方块下落到地面。这个程序已经包含了后续机器人仿真的几个核心步骤：读取 XML、创建仿真状态、调用 `mj_step()` 推进物理仿真，以及调用 Viewer 显示当前结果。

---

## 4. MjModel 与 MjData

阅读 MuJoCo Python 程序时，经常会看到下面两行：

```
model = mujoco.MjModel.from_xml_path("scene.xml")
data = mujoco.MjData(model)
```

**MjModel** 保存编译后的模型信息，包括刚体、关节、执行器、质量、惯量、碰撞几何、仿真参数等。对于正常运行中的机器人，这些信息大部分保持固定，因此可以把它理解为当前仿真系统的结构和参数。

**MjData** 保存仿真运行过程中的状态和中间计算结果，例如当前位置、当前速度、控制输入和仿真时间。随着 `mj_step()` 不断执行，**MjData** 中的大量内容会持续变化。

可以使用下面的程序观察一些基本信息：

```
print("nq =", model.nq)
print("nv =", model.nv)
print("nu =", model.nu)

print("qpos =", data.qpos)
print("qvel =", data.qvel)
print("ctrl =", data.ctrl)
print("time =", data.time)
```

其中：

| 名称                     | 含义                                  |
|------------------------|-------------------------------------|
| <code>model.nq</code>  | <code>qpos</code> 的维度               |
| <code>model.nv</code>  | <code>qvel</code> 的维度               |
| <code>model.nu</code>  | 控制输入 <code>ctrl</code> 的维度，也就是执行器数量 |
| <code>data.qpos</code> | 当前广义位置                              |
| <code>data.qvel</code> | 当前广义速度                              |

| 名称                     | 含义        |
|------------------------|-----------|
| <code>data.ctrl</code> | 当前执行器控制输入 |
| <code>data.time</code> | 当前仿真时间    |

对于后续四足机器人程序，`MjModel` 和 `MjData` 会反复出现。遇到陌生代码时，可以先判断某个量来自模型参数还是来自当前运行状态，这通常能够帮助快速理解程序结构。

## 5. `mj_step()` 与仿真循环

MuJoCo 中最常见的推进函数是：

```
mujoco.mj_step(model, data)
```

每调用一次，MuJoCo 会根据当前模型、当前状态和控制输入计算一个仿真时间步之后的新状态。模型中的时间步长由 `option` 的 `timestep` 决定，例如：

```
<option timestep="0.002"/>
```

表示每个物理仿真步对应：

```
0.002 s
```

也就是理论上的 500 Hz 物理更新频率。

很多机器人控制程序都会采用类似下面的结构：

```
while running:
    # 1. 读取机器人状态

    # 2. 计算控制量

    # 3. 写入 data.ctrl

    # 4. 推进物理仿真
    mujoco.mj_step(model, data)

    # 5. 更新可视化
```

需要注意，物理仿真频率和控制器频率可以不同。例如 MuJoCo 每 0.001 s 推进一次，也就是 1000 Hz，而控制器每 0.02 s 更新一次，也就是 50 Hz。这种情况下，每计算一次新的控制量，可以保持该控制量并连续执行 20 个物理仿真步。

```
控制器更新一次
↓
保持当前控制量
↓
mj_step × 20
↓
控制器再次更新
```

后续四足机器人的强化学习策略通常也会采用类似方式，因此需要从现在开始区分“仿真一步”和“控制一次”这两个概念。

## 6. MJCF 的基本结构

MuJoCo 原生使用 MJCF（MuJoCo Modeling Language）描述模型。MJCF 使用 XML 语法，因此模型通常保存为 `.xml` 文件。一个较完整的 MJCF 文件可能包含大量参数，但目前只需要掌握几个最常用的部分：

```
<mujoco model="example">
```

```

<compiler/>
<option/>

<asset>
    ...
</asset>

<worldbody>
    ...
</worldbody>

<actuator>
    ...
</actuator>

<sensor>
    ...
</sensor>
</mujoco>

```

这些部分承担的作用不同。`compiler` 设置模型解析和编译相关选项，`option` 设置仿真时间步长、重力和求解参数等运行选项，`asset` 保存 mesh、材质和纹理等资源，`worldbody` 描述世界中的刚体结构，`actuator` 描述执行器，`sensor` 描述需要从仿真中读取的传感器量。

本阶段需要优先看懂 `worldbody` 和 `actuator`。四足模型的大部分机械结构都会位于 `worldbody` 中，而真正向关节施加控制作用的部分通常定义在 `actuator` 中。

## 7. body、joint 与 geom

MJCF 中的机器人结构按照刚体层级组织。下面是一个简化的单关节结构：

```

<worldbody>
  <body name="base" pos="0 0 0.5">
    <geom type="box" size="0.2 0.1 0.05"/>

```

```
<body name="link" pos="0 0 -0.1">
  <joint
    name="joint1"
    type="hinge"
    axis="0 1 0"
  />
  <geom
    type="capsule"
    size="0.03 0.2"
  />
</body>
</worldbody>
```

**body** 表示刚体以及该刚体所使用的局部坐标系。MJCF 通过嵌套的 **body** 建立机器人运动学树，子 **body** 会随着父 **body** 一起运动。四足机器人的机身、大腿、小腿等都可以对应不同的 **body**。

**joint** 定义当前 **body** 相对于父 **body** 允许怎样运动。机器人腿部最常见的是 **hinge**，它表示绕指定轴旋转。常见关节类型还包括 **slide** 和 **free**，分别用于直线运动和六自由度自由运动。

**geom** 描述几何形状，可以参与碰撞，也可以用于显示。一个 **body** 中可以包含多个 **geom**，因此读取模型时不要简单地把 **body** 数量和 **geom** 数量对应起来。实际机器人模型中还经常使用 mesh 几何来表示复杂外形。

对于四足机器人，可以先按照下面的结构理解：

```
base body
├── front-left hip body
│   ├── thigh body
│   └── calf body
├── front-right hip body
│   └── ...
├── rear-left hip body
│   └── ...
└── rear-right hip body
    └── ...
```

各个 `body` 之间通过 `joint` 形成自由度，再由 `geom` 提供碰撞和外观几何。

## 8. 自由基座与 `freejoint`

机器人直接放在地面上进行动力学仿真时，机身需要能够整体平移和旋转，因此通常会在基座 `body` 中定义：

```
<freejoint/>
```

自由关节提供三维平移和三维旋转共 6 个速度自由度。它在 `qpos` 中使用 7 个数保存基座位姿，其中前三个是位置，后四个表示姿态四元数；在 `qvel` 中使用 6 个数保存线速度和角速度。

因此，对于一个带自由基座并具有 12 个单自由度关节的四足机器人，常见的维度关系是：

```
qpos: 7 + 12 = 19
qvel: 6 + 12 = 18
```

这也是为什么 `len(data.qpos)` 和 `len(data.qvel)` 往往不同，也不能直接把 `qpos` 的长度理解为关节数量。后续编写强化学习部署代码时，如果需要从 `qpos` 中提取 12 个关节角，就必须先弄清自由基座占用了哪些位置。

可以通过：

```
print(model.nq)
print(model.nv)
```

确认当前模型的实际维度。面对别人提供的模型时，不要仅根据机器人“有 12 个电机”去猜测 `qpos` 和 `qvel` 的长度。

## 9. `inertial`、质量与碰撞

动力学仿真需要知道每个刚体的质量、质心和惯量等参数。MJCF 中可以显式使用 `inertial` 描述这些信息，也可以根据 `geom` 的质量或密度让 MuJoCo 计算相应惯性参数。

例如：

```
<body name="link">
  <inertial
    pos="0 0 -0.1"
    mass="1.2"
    diaginertia="0.01 0.02 0.02"
  />
</body>
```

目前不要求推导惯性矩阵，但需要知道质量、质心和惯量会直接影响机器人运动。如果模型在仿真中表现出明显异常，例如轻微接触就快速弹飞、某条腿运动得异常剧烈，除了检查控制器，还应该检查质量、惯量、碰撞几何和关节参数。

机器人外观模型和碰撞模型也可能采用不同的 `geom`。为了让仿真稳定并减少计算量，工程中经常使用较简单的碰撞几何近似复杂外形，例如用 `box`、`capsule` 和 `sphere` 近似腿部和机身。外观 `mesh` 适合显示，但直接使用非常复杂的 `mesh` 进行碰撞计算可能增加计算量和调试难度。

## 10. URDF 与 MJCF

ROS 和机器人开源项目中经常使用 URDF (Unified Robot Description Format) 描述机器人。URDF 中常见的概念包括 `link`、`joint`、`visual`、`collision` 和 `inertial`，它能够很好地描述机器人的基本结构。

MJCF 的组织方式有所不同。URDF 使用若干 `link` 和 `joint` 描述父子关系，MJCF 则通过嵌套的 `body` 直接构成运动学树。两者都能表示机器人结构，但 MJCF 还提供了大量与 MuJoCo 仿真相关的功能，例如默认参数、执行器、传感器和其他仿真配置。

需要特别说明的是，当前版本的 MuJoCo 已经能够解析 URDF，并将其编译为内部模型。因此，从软件能力上看，可以直接让 MuJoCo 读取符合要求的 URDF。培训任务仍然要求大家完成 **URDF → MJCF** 的转换和整理，因为后续需要阅读和修改 MJCF、加入执行器、组织场景，并逐渐接触 MuJoCo 特有的模型参数。显式完成这一过程也有助于理解两种机器人描述方式之间的对应关系。

可以先形成下面的概念：

```
URDF
link + joint
  ↓
解析 / 转换 / 整理
  ↓
MJCF
body + joint + geom + actuator + ...
  ↓
MuJoCo 编译
  ↓
MjModel
```

转换后的 MJCF 仍然需要检查。尤其要确认 mesh 路径、机器人初始姿态、关节轴、关节范围、质量惯量、碰撞体和执行器等是否符合预期。模型能够成功打开，只能说明 XML 能够被解析，无法自动保证动力学和控制配置全部正确。

## 11. 场景与机器人模型

实际项目通常会把机器人和环境分开组织。机器人模型主要描述机器人本体，场景文件再加入地面、灯光、相机和环境物体。这样同一个机器人可以被放入不同场景中使用，而不需要复制大量机器人 XML。

一个简单的工程可以组织为：

```
mujoco_project/  
├── models/  
│   └── robot.xml  
├── scenes/  
│   └── flat_scene.xml  
├── scripts/  
│   └── simulate.py  
└── README.md
```

MJCF 支持通过 `include` 组合多个 XML 文件。例如主场景可以包含其他模型文件：

```
<include file="../../models/robot.xml"/>
```

`include` 在解析阶段会把被包含文件中的 XML 元素合并到当前模型中，因此最终仍然需要形成一个合法的 MJCF 模型。实际开源项目的文件组织方式可能更加复杂，但阅读时可以先把它们归纳为“机器人本体”“环境场景”“控制程序”三个部分。

平坦地面通常可以用：

```
<geom  
  name="floor"  
  type="plane"  
  size="5 5 0.1"  
>
```

表示。对于当前培训任务，先学会加载一个简单平地场景即可。后续进入强化学习训练时，地形会逐渐扩展到坡面、台阶、随机高度场等更复杂形式。

---

## 12. joint 与 actuator

定义了关节以后，模型就拥有相应的运动自由度，但控制程序还需要通过执行器向这些自由度施加作用。MJCF 中的执行器通常位于：

```
<actuator>
...
</actuator>
```

对于当前四足培训，最需要掌握的是 `motor`：

```
<actuator>
  <motor
    name="joint1_motor"
    joint="joint1"
    gear="1"
  />
</actuator>
```

`motor` 是 MuJoCo 提供的直接驱动执行器。上面的配置把执行器连接到 `joint1`。在最简单的一对一旋转关节、`gear="1"` 的情况下，可以把写入 `data.ctrl` 的控制量理解为该执行器的直接驱动输入，并用于产生对应的关节力矩。实际模型如果设置了不同的 `gear`、控制范围或更复杂的传动关系，控制量和最终关节力矩之间还会存在相应变换。

Python 中可以写：

```
data.ctrl[0] = 1.0
```

也可以一次设置所有执行器：

```
data.ctrl[:] = 0.0
```

`data.ctrl` 的长度由：

```
model.nu
```

决定。因此，一个具有 12 个独立电机执行器的四足模型通常会看到：

```
model.nu = 12
```

但仍应以实际模型为准。

MuJoCo 还提供 `position`、`velocity` 等执行器形式。它们内部包含不同的反馈和增益关系。本阶段重点放在直接驱动的 `motor` 上，后面学习 PD 控制时，再进一步分析位置、速度、力矩之间的关系。

## 13. `qpos`、`qvel` 与 `ctrl`

对于刚开始接触 MuJoCo 的程序，最值得先熟悉的三个数组是：

```
data.qpos  
data.qvel  
data.ctrl
```

`qpos` 保存广义位置，内容可能包括机器人基座位置、基座姿态以及各个关节的位置。`qvel` 保存对应的广义速度，其中自由基座姿态的速度表示方式与 `qpos` 不同，因此两者维度可能不同。`ctrl` 保存执行器输入，其维度与执行器数量相关。

可以使用：

```
print(data.qpos.shape)  
print(data.qvel.shape)  
print(data.ctrl.shape)
```

直接检查数组维度。

对于一个自由基座四足机器人，如果关节排列与模型设计一致，程序中可能出现类似：

```
base_position = data.qpos[0:3]
base_quaternion = data.qpos[3:7]
joint_position = data.qpos[7:]
```

这段写法只有在你已经确认模型的自由度排列后才能安全使用。更复杂的模型可能还包含额外关节，因此阅读陌生代码时应先检查 XML 和模型信息，再决定数组切片方式。

MuJoCo 的 Python 接口还支持根据名称访问模型对象。工程代码中如果大量依赖固定索引，模型结构一旦修改就可能导致索引变化，因此后续可以逐步学习通过关节、执行器和传感器名称查找对应 ID。当前阶段先理解索引和模型结构之间存在明确对应关系即可。

## 14. 零力矩与机器人静止

如果执行：

```
data.ctrl[:] = 0.0
```

控制程序没有向执行器提供主动驱动力矩，但机器人仍然受到重力、接触力、关节阻尼以及模型中其他物理参数的影响。如果机器人初始时悬在空中，它会下落；如果初始姿态较高且没有主动支撑，腿部也会在重力作用下发生运动。

因此，“零力矩”不能直接理解为“机器人一定保持原姿态”。是否能够最终保持静止，需要结合初始姿态和接触状态判断。一个合理的趴卧姿态可以让机身或腿部与地面形成稳定接触，经过短暂运动后可能达到稳定状态，而站立姿态通常需要控制器持续提供支撑力矩。

当前任务要求将机器人放在平坦场景中，并在执行器输入为零的情况下形成合理的静止状态。完成任务时应观察机器人是否持续漂移、穿透地面、剧烈弹跳或产生异常关节运动。如果出现这些现象，应继续检查初始位置、碰撞体、质量惯量、关节配置和接触参数，而不要只修改控制程序。

---

## 15. Viewer 与仿真显示

MuJoCo Python 包自带 Viewer。可以直接通过命令启动：

```
python3 -m mujoco.viewer
```

也可以加载一个 MJCF 文件：

```
python3 -m mujoco.viewer --mjcf=/path/to/model.xml
```

在 Python 程序中，培训阶段更常使用：

```
mujoco.viewer.launch_passive(model, data)
```

`launch_passive` 会启动可视化窗口，同时让 Python 主程序继续执行，因此适合自己编写控制循环。主程序需要主动调用：

```
mujoco.mj_step(model, data)  
viewer.sync()
```

`mj_step()` 更新物理状态，`viewer.sync()` 将当前状态同步到显示窗口。Viewer 的显示刷新和物理仿真属于两个不同概念，程序即使没有每个物理步都刷新画面，也可以继续推进物理计算。

调试时，Viewer 很适合快速检查模型位置、关节运动、接触情况和几何体是否明显异常。遇到程序问题时，应同时观察终端信息和仿真窗口，不要只看机器人“有没有动”。

---

## 16. 一个基础的力矩控制程序结构

在加入执行器以后，一个基础的 MuJoCo 控制程序可以整理为：

```
import time

import mujoco
import mujoco.viewer

model = mujoco.MjModel.from_xml_path("scene.xml")
data = mujoco.MjData(model)

with mujoco.viewer.launch_passive(model, data) as viewer:
    while viewer.is_running():
        step_start = time.time()

        # 读取当前状态
        qpos = data.qpos.copy()
        qvel = data.qvel.copy()

        # 计算控制量
        torque = 0.0

        # 写入执行器控制输入
        if model.nu > 0:
            data.ctrl[:] = torque

        # 推进仿真
        mujoco.mj_step(model, data)

        # 更新 Viewer
        viewer.sync()

        # 让显示速度大致接近真实时间
        time_left = model.opt.timestep - (time.time() - step_start)
        if time_left > 0:
            time.sleep(time_left)
```

这里把程序分成了“读取状态、计算控制量、写入执行器、推进仿真、更新显示”几个部分。后续控制算法复杂以后，仍然可以保持类似结构，只需要把计算控制量的部分替换成 PD 控制器、状态机或强化学习策略。

对于培训阶段的小程序，可以先写在一个文件中。随着功能增加，应逐步考虑把模型加载、控制器、机器人接口和主循环拆分到不同模块中。第二次培训中学习到的类、封装、继承和组合等思想，会在这里逐渐和机器人程序结合起来。

## 17. 从小程序过渡到工程结构

如果所有逻辑都放在一个 `simulate.py` 中，几十行代码时还可以阅读，但后面加入状态读取、传感器、控制器、日志和通信以后，文件会快速变得难以维护。因此可以逐步整理成：

```
mujoco_project/
├── models/
│   └── robot.xml
├── scenes/
│   └── flat_scene.xml
├── src/
│   ├── robot.py
│   ├── controller.py
│   └── simulator.py
├── main.py
└── README.md
```

例如 `simulator.py` 负责 MuJoCo 模型和仿真循环，`robot.py` 负责机器人关节和执行器相关信息，`controller.py` 负责根据状态生成控制量，`main.py` 负责组合这些模块并启动程序。具体如何拆分没有唯一模板，需要根据项目规模和功能确定。

这里可以开始把前面学习的面向对象思想应用到实际问题中。例如，可以用一个类保存模型和 `MjData`，用成员函数实现状态读取和仿真推进；控制器再通过统一接口接收状态并返回控制量。当前阶段只需要形成“程序结构应该随着功能增长而整理”的意识，不需要一开始就设计复杂的类层级。

## 18. 如何阅读 unitree\_mujoco 一类开源项目

直接打开一个完整机器人仓库时，文件数量通常很多。从目录第一行一路阅读到最后一行效率很低，更合适的方法是先找到程序的主流程。

建议按照下面的顺序阅读：

```
README
  ↓
运行命令
  ↓
程序入口
  ↓
模型在哪里加载
  ↓
MjModel / MjData 在哪里创建
  ↓
主循环在哪里
  ↓
状态在哪里读取
  ↓
data.ctrl 在哪里写入
  ↓
mj_step 在哪里执行
  ↓
其他模块分别负责什么
```

首先通过 README 确认项目用途和启动方式，再从入口文件沿函数调用关系向下查找。看到陌生类时，先确认它在整个运行流程中承担什么职责，再阅读类内部细节。这样更容易建立程序的整体结构。

参考开源代码时，应重点观察别人如何组织工程、如何表示机器人状态、如何划分仿真与控制模块，以及哪些参数被放入配置文件。可以借鉴这些设计改进自己的程序，但最终需要理解自己保留的每一部分核心逻辑。

## 19. 常见问题与排查顺序

### 19.1 ModuleNotFoundError: No module named 'mujoco'

先检查：

```
which python3  
python3 -m pip show mujoco
```

如果 `pip` 安装使用的 Python 与当前运行脚本的 Python 不一致，就可能出现已经安装但无法导入的情况。

### 19.2 XML 无法加载

MuJoCo 通常会给出 XML 解析或模型编译错误，并指出大致位置。优先阅读错误信息，检查标签、属性名、文件路径和被引用的 mesh 是否存在。修改模型后重新加载，不要一次改动大量参数，否则很难判断具体是哪一项导致问题。

### 19.3 mesh 找不到

先检查 XML 中的相对路径，再确认当前模型文件和资源目录之间的关系。使用：

```
pwd  
ls  
find . -name "*.stl"  
find . -name "*.obj"
```

可以帮助确认文件实际位置。模型中的相对资源路径通常按照模型文件和编译器相关设置进行解析，因此复制 XML 时也要注意把资源目录一起保留。

## 19.4 机器人加载后直接掉下去

先确认机器人是否有自由基座、初始高度是否合理，以及地面是否存在。随后检查机器人初始姿态和关节角。对于零力矩模型，站立状态失去支撑后下落属于正常动力学结果。

## 19.5 机器人剧烈抖动或弹飞

先把问题拆开检查。可以暂时令所有 `ctrl` 为零，确认异常是否仍然存在；再检查初始模型是否互相穿透、碰撞体是否过大、质量惯量是否异常以及关节范围是否合理。如果零控制时模型已经不稳定，应先处理模型和接触问题，再继续调控制器。

## 19.6 修改 `data.ctrl` 后关节没有明显运动

确认模型中是否真的定义了 `actuator`，并检查：

```
print(model.nu)
print(data.ctrl)
```

随后确认 `actuator` 是否连接到了预期关节，控制范围、`gear` 和关节阻尼等参数是否合理。单纯存在 `joint` 并不能保证 `data.ctrl` 能直接控制它。

## 20. 调试时建议观察的信息

机器人仿真出现问题时，可以先输出少量关键数据：

```
print("time:", data.time)
print("qpos:", data.qpos)
print("qvel:", data.qvel)
print("ctrl:", data.ctrl)
```

如果数组太长，可以只查看一部分：

```
print("base position:", data.qpos[:3])
print("joint position:", data.qpos[7:])
```

也可以先查看模型维度：

```
print("nq:", model.nq)
print("nv:", model.nv)
print("nu:", model.nu)
```

调试时尽量每次只确认一个问题。例如先确认模型是否成功加载，再确认初始姿态，再确认执行器数量，再确认控制量是否写入，最后再观察机器人运动。把多个未知问题混在一起修改，往往会增加排查难度。

## 21. 本阶段需要掌握的能力

完成本文后，应能够解释 MuJoCo 在机器人开发中的基本作用，并能够独立完成一个简单仿真程序。对于具体 API 不要求全部记忆，但需要知道去哪里查找，并能够理解常见代码的含义。

本阶段至少应达到以下程度：

1. 能够安装并验证 MuJoCo Python 包；
2. 能够使用 `MjModel.from_xml_path()` 加载模型；
3. 能够创建 `MjData` 并使用 `mj_step()` 推进仿真；
4. 能够理解 `body`、`joint`、`geom`、`actuator` 的基本作用；
5. 能够说明 URDF 和 MJCF 的基本区别与联系；
6. 能够理解自由基座对 `qpos` 和 `qvel` 维度的影响；
7. 能够读取 `qpos`、`qvel` 并向 `ctrl` 写入控制量；
8. 能够建立一个平坦地面场景并加载机器人模型；
9. 能够理解零力矩状态下机器人仍然受到重力和接触作用；
10. 能够根据报错、模型结构和关键状态量进行基础排查；

11. 能够从 README、程序入口和主循环开始阅读一个 MuJoCo 开源项目。

本阶段暂时不要求理解 MuJoCo 的接触求解器、约束方程、雅可比矩阵、逆动力学和高级执行器模型。这些内容会随着机器人动力学、控制和强化学习训练继续展开。

## 22. 建议的学习顺序

阅读本文时，建议实际运行代码，不要只阅读文字。可以按照下面的顺序完成：

```
安装 MuJoCo
  ↓
运行 falling box 示例
  ↓
理解 MjModel / MjData
  ↓
查看 qpos / qvel
  ↓
阅读 MJCF 的 body / joint / geom
  ↓
理解 freejoint
  ↓
加入 actuator
  ↓
修改 data.ctrl
  ↓
拆分 robot 与 scene
  ↓
再进入四足机器人模型
```

如果某一步出现问题，应优先把当前最小示例调通，再继续增加新的结构。机器人模型包含很多刚体、关节和资源文件，一开始直接在完整四足模型中排查所有问题会明显增加难度。

## 23. 进一步阅读

后续需要查询具体标签或 Python 接口时，优先参考 MuJoCo 官方文档：

- MuJoCo Documentation: <https://mujoco.readthedocs.io/>
- Python API: <https://mujoco.readthedocs.io/en/latest/python.html>
- Modeling: <https://mujoco.readthedocs.io/en/latest/modeling.html>
- XML Reference: <https://mujoco.readthedocs.io/en/latest/XMLreference.html>
- Programming: <https://mujoco.readthedocs.io/en/latest/programming/>

查阅官方文档时，不需要一次读完所有内容。遇到具体问题再定位对应章节，例如 actuator 配置查 XML Reference，Python Viewer 查 Python 文档，模型组织和 MJCF 机制查 Modeling。随着项目推进，反复查阅这些文档会比记忆大量参数更有效。